

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico Parte 1



3 de diciembre de 2024

Nombre	Padrón
Maximo Giovanettoni	110303
Franco Bossi	111122
Luca Saluzzi	108088
Agustin Conti	107800
Gabriel Peralta	101767

1. Algoritmo

El problema presentado es el siguiente: Se tiene una fila de n monedas con diferentes valores, en cada turno uno de los dos jugadores elige entre agarrar la primera moneda de la fila o la última. El que acumula más valor es el ganador. En este caso en particular, si bien hay dos jugadores, solo Sophia elige monedas. Es decir, que en su turno toma una moneda para él y en el próximo una moneda para el jugador rival.

Se nos pedía diseñar un algoritmo greedy que haga que el jugador 1 (Sophia) siempre gane, terminando siempre con un valor total acumulado mayor al del jugador 2 (Mateo).

El algoritmo planteado es el siguiente:

1.1. Jugador 1 elige moneda

El jugador1, en su turno, debe elegir la moneda más grande, por lo que compara la del principio y la del final y agarra aquella de mayor valor. En nuestra implementación, en vez de "sacar.esa moneda de la pila (que vendría a ser equivalente a hacer pop de la lista), simplemente avanza el índice correspondiente, que puede variar según si eligió la primera o la última moneda.

```
1 if (jugador == JUGADOR1 and monedas[izq] > monedas[der]):
2     return monedas[izq], izq + 1, der
3 else:
4     return monedas[der], izq, der - 1
```

1.2. Jugador 1 elige la moneda del Jugador 2

En el turno del Jugador 2, es el Jugador 1 el que vuelve a elegir la moneda, pero en este caso para su rival. Es por esto que deberá nuevamente comparar la moneda inicial con la final y agarrar la de menor valor. Al igual que en el caso anterior, se avanza el índice correspondiente.

```
1 if (jugador == JUGADOR2 and monedas[izq] < monedas[der]):
2     return monedas[izq], izq + 1, der
3 else:
4     return monedas[der], izq, der - 1
```

1.3. Loop

Estos dos pasos anteriores se repetirán hasta que la posición inicial sea mayor a la final, es decir, hasta que los índices final e inicial estén en la misma posición.

1.4. Algoritmo completo y complejidad algorítmica

En este caso particular, Sophia es el jugador 1 y Mateo el jugador 2

```
1 SOPHIA = 0
2 MATEO = 1
3
4 def elegir_moneda(monedas, jugador, izq, der):
5     if (jugador == SOPHIA and monedas[izq] > monedas[der]) or (jugador == MATEO and
6         monedas[izq] < monedas[der]):
7         return monedas[izq], izq + 1, der
8     else:
9         return monedas[der], izq, der - 1
10
11 def greedy_monedas(monedas):
12     izq, der = 0, len(monedas) - 1
13     puntos_sophia, puntos_mateo = 0, 0
14     turno_sophia = True
```

```
15 while izq <= der:
16     jugador = SOPHIA if turno_sophia else MATEO
17     moneda, izq, der = elegir_moneda(monedas, jugador, izq, der)
18     if turno_sophia:
19         puntos_sophia += moneda
20     else:
21         puntos_mateo += moneda
22     turno_sophia = not turno_sophia
23
24 return puntos_sophia, puntos_mateo
```

Tal y como se aprecia en el código, este algoritmo trabaja en $O(n)$ ya que se recorre una sola vez cada posición de la lista de monedas, avanzando el índice cuando es necesario. Esto se podrá comprobar más adelante en conjunto con los gráficos correspondientes.

2. Optimalidad del Algoritmo

2.1. Razonamiento Inductivo

Comprobaremos entonces la optimalidad del algoritmo mediante un razonamiento inductivo:

2.1.1. Base inductiva

Con una sola moneda ($n=1$) Sophia gana porque empieza ella y la elige

Monedas: 5

Sophia: [5]

Mateo: []

2.1.2. Paso inductivo

Al agregar una moneda al problema Sophia sigue ganando porque Mateo agarra la menor (desestimamos el caso de una cantidad par de monedas de mismo valor ya que es inevitablemente empate).

Monedas:5;1

Sophia: [5]

Mateo: [1]

Si seguimos agregando monedas va a ser lo mismo ya que Sophia va elegir la mayor y Mateo seguirá eligiendo la menor, sumando siempre menos que Sophia para cualquier n .

Monedas: 5;1;5

Sophia: [5;5]

Mateo: [1]

Monedas: 5;1;5;5

Sophia: [5;5]

Mateo: [5;1]

Con esta lógica afirmamos que Sophia siempre va a ganarle a Mateo independientemente de la cantidad de monedas o sus valores.

2.2. Para n monedas pares y n monedas impares

2.2.1. Pares

$n, n+1, n+2, n+3$ -¿S: $n+3$ $n+2$ M: n $n+1$
 $n, n+1, n+3, n+2$ -¿S: $n+2$ $n+3$ M: n $n+1$
 $n, n+2, n+1, n+3$ -¿S: $n+3$ $n+2$ M: n $n+1$
 $n, n+2, n+3, n+1$ -¿S: $n+1$ $n+3$ M: n $n+2$
 $n, n+3, n+1, n+2$ -¿S: $n+2$ $n+3$ M: n $n+2$
 $n, n+3, n+2, n+1$ -¿S: $n+1$ $n+3$ M: n $n+2$
 $n+1, n, n+2, n+3$ -¿S: $n+3$ $n+2$ M: $n+1$ n
 $n+1, n, n+3, n+2$ -¿S: $n+2$ $n+3$ M: $n+1$ n
 $n+1, n+2, n, n+3$ -¿S: $n+3$ $n+2$ M: $n+1$ n
 $n+1, n+2, n+3, n$ -¿S: $n+1$ $n+3$ M: $n+2$ n
 $n+1, n+3, n, n+2$ -¿S: $n+2$ $n+3$ M: $n+1$ n
 $n+1, n+3, n+2, n$ -¿S: $n+1$ $n+3$ M: $n+2$ n
 $n+2, n, n+1, n+3$ -¿S: $n+3$ $n+2$ M: $n+1$ n
 $n+2, n, n+3, n+1$ -¿S: $n+2$ $n+3$ M: $n+1$ n
 $n+2, n+1, n, n+3$ -¿S: $n+3$ $n+2$ M: $n+1$ n
 $n+2, n+1, n+3, n$ -¿S: $n+2$ $n+3$ M: n $n+1$
 $n+2, n+3, n, n+1$ -¿S: $n+2$ $n+3$ M: $n+1$ n
 $n+2, n+3, n+1, n$ -¿S: $n+2$ $n+3$ M: n $n+1$
 $n+3, n, n+1, n+2$ -¿S: $n+3$ $n+2$ M: n $n+1$
 $n+3, n, n+2, n+1$ -¿S: $n+3$ $n+2$ M: n $n+1$
 $n+3, n+1, n, n+2$ -¿S: $n+3$ $n+2$ M: $n+1$ n
 $n+3, n+1, n+2, n$ -¿S: $n+3$ $n+2$ M: n $n+1$
 $n+3, n+2, n, n+1$ -¿S: $n+3$ $n+2$ M: $n+1$ n
 $n+3, n+2, n+1, n$ -¿S: $n+3$ $n+2$ M: n $n+1$

2.2.2. Impares

$n, n+1, n+2$ -¿S: $n+2$ $n+1$ M: n
 $n+1, n+2, n$ -¿S: $n+1$ $n+2$ M: n
 $n, n+2, n+1$ -¿S: $n+1$ $n+2$ M: n
 $n+2, n, n+1$ -¿S: $n+1$ $n+2$ M: n
 $n+1, n, n+2$ -¿S: $n+2$ $n+1$ M: n
 $n+2, n+1, n$ -¿S: $n+2$ $n+1$ M: n

Podríamos decir que para todo conjunto par o impar de números, Sophia siempre ganara sin importar el orden en el que se encuentren los números.

3. Análisis de variabilidad y mediciones

Para comparar como afectan los distintos valores de las monedas (no la cantidad, si no los valores en si), medimos varias ejecuciones con sets de datos de alta y baja variabilidad.

```
1 VARIABILIDAD_BAJA = 10
2 VARIABILIDAD_ALTA = 1000000
3
4 # Tamaño del array
5 x = np.linspace(1000, 10_000_000, 25).astype(int)
6
7 # Generador de monedas aleatorias
8 def get_random_cant_monedas(s, var):
9     return [random.randint(1, var) for _ in range(s)]
10
11 # Medimos los tiempos de ejecución
12 results_low_var = time_algorithm(greedy_monedas, x, lambda s: [
13     get_random_cant_monedas(s, VARIABILIDAD_BAJA)])
14 results_high_var = time_algorithm(greedy_monedas, x, lambda s: [
15     get_random_cant_monedas(s, VARIABILIDAD_ALTA)])
```

Tal y como se puede observar en los siguientes gráficos. El algoritmo se comporta en forma $O(n)$, con un ajuste muy bueno con respecto a las rectas en ambos casos y con un error pico del 0.2 en los casos de mayor variabilidad. Podríamos concluir entonces que, para los casos de mayor variabilidad, el error se maximiza y se diferencia para tamaños de arrays mayores.

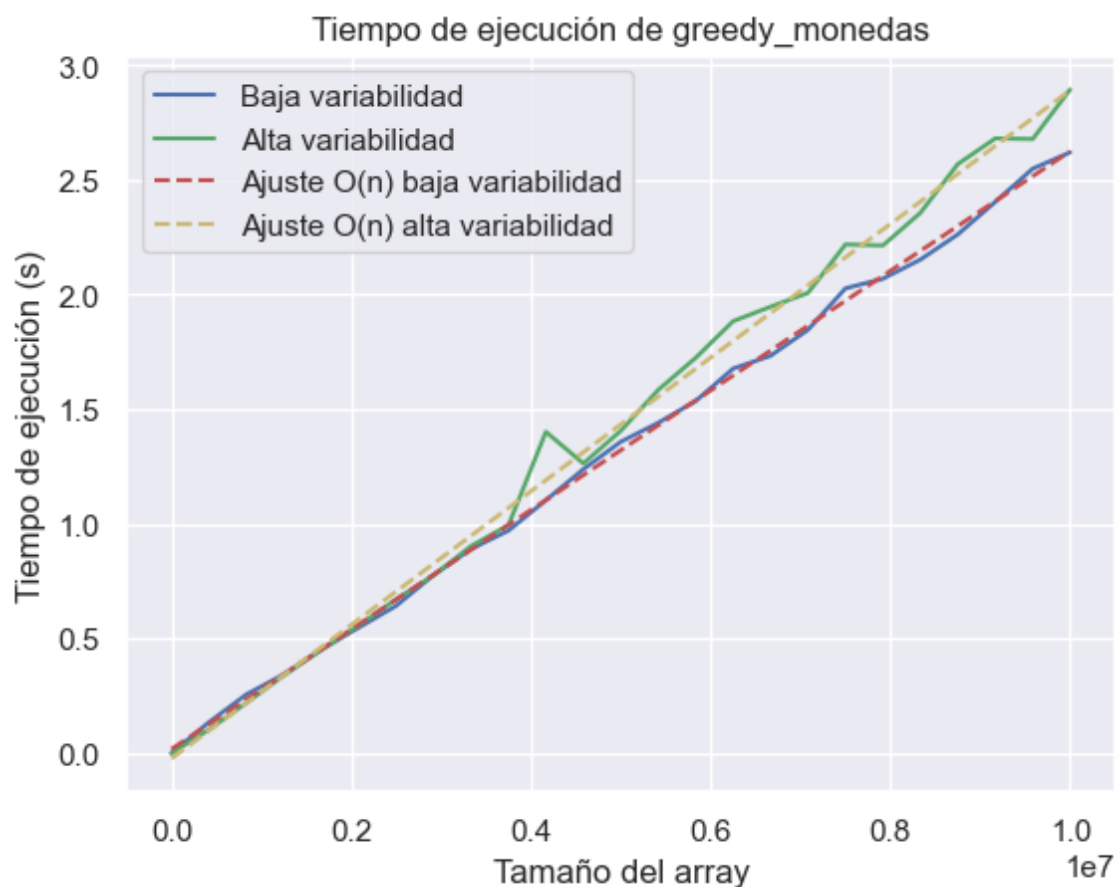


Figura 1: Tiempo de Ejecución del algoritmo con alta y baja variabilidad comparado con rectas de ajuste

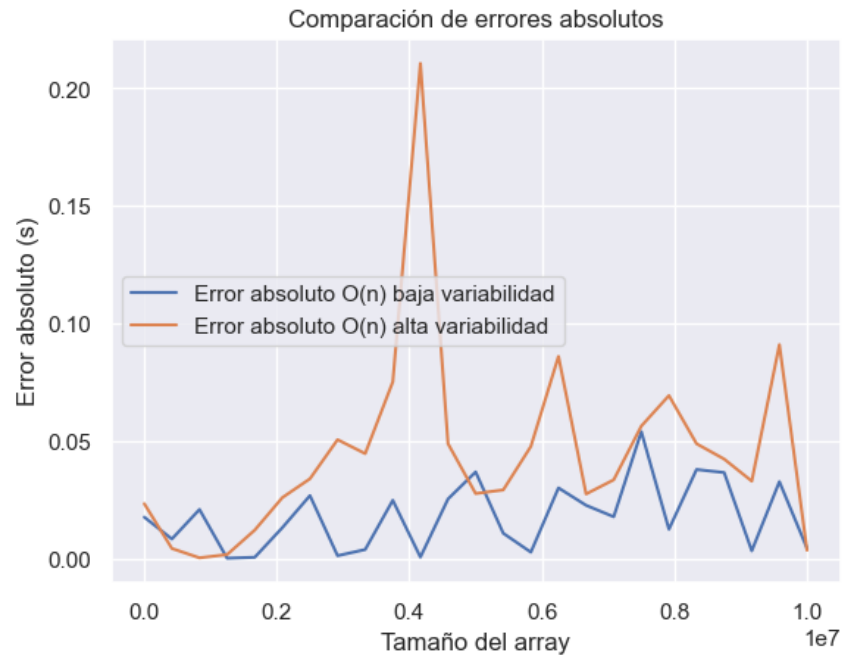


Figura 2: Errores absolutos del algoritmo según tamaño del array con alta y baja variabilidad

Notamos que suele haber mas error con variabilidad alta, pero no siempre es el caso. Lo probamos 20 veces con los parámetros mostrados y resultó en que el 60 % de las veces variabilidad alta tenía más error.